

CHECK WHAT YOU LEARNT

Check Yourself - Unit 4**A. Choose the correct answer**

1. A piezoelectric disc can act as
(a) only a speaker (b) only a microphone (c) both, in opposite directions (d) neither
2. 40 kHz is chosen because it is
(a) the loudest (b) just above human hearing (c) the cheapest (d) visible
3. An object 2 cm from the sensor gives nonsense because of the
(a) beam angle (b) blind zone (c) gain error (d) timeout
4. A reading that drifts further out as distance grows indicates
(a) offset error (b) gain error (c) random error (d) no error
5. A single wild reading affects
(a) the median more than the mean (b) the mean more than the median (c) both equally (d) neither
6. Sound travels faster when the air is
(a) cooler (b) warmer (c) drier (d) still

B. Fill in the blanks

- 1 The beam of an HC-SR04 spreads about _____ degrees.
- 2 Readings below about _____ cm fall inside the blind zone.
- 3 A constant shift in every reading is called _____ error.
- 4 The median filter works by _____ the readings and taking the middle one.

C. Answer in one or two lines

- 1 Write down your gain and offset, and the two data points you derived them from.
- 2 Explain why a tilted target gives a poor or missing reading.
- 3 State the working range of your sensor and justify it from your data.
- 4 Explain, with your own numbers, why you chose a median or a mean filter.

UNIT 5**Capstone Project: Maze Solver Robot**

Sessions 12, 13, 14 and 15

The capstone brings everything together: three sensors, a driver stage, a state machine, and memory. The robot must reach the exit of a maze it has never seen, and then do it again faster using what it learnt on the first run.

By the end of this unit you will be able to:

- Implement and document a navigation algorithm end to end.
- Design a state machine and explain why it beats a chain of if statements.
- Store a route in memory and simplify it.
- Defend design trade-offs with test data in a technical viva.

SESSION 12

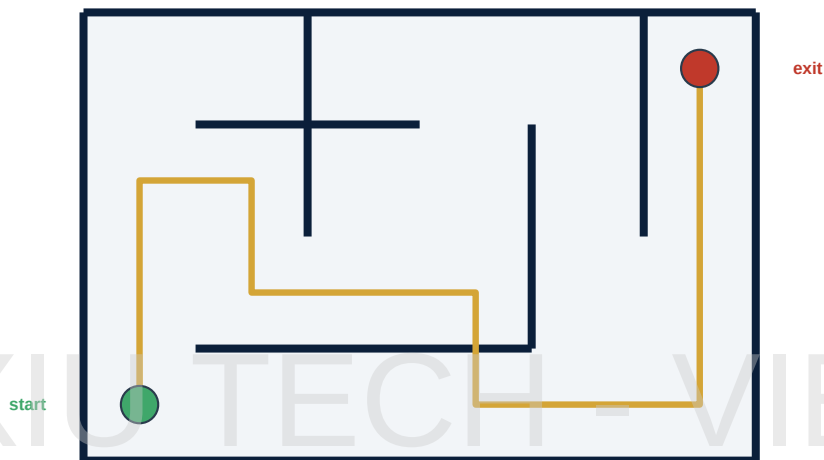
one class

Today you will

Solving a maze sounds like it needs intelligence. It does not. A rule discovered centuries before computers existed will do it: keep one hand on the wall and walk. You will trace every wall connected to the one you started on, and if the exit is on that same connected wall, you will certainly reach it.

The left-hand rule

Keep your left hand on the wall and walk. You will always reach the exit.



It is not the shortest route - but it never fails, and it needs no map at all.

Figure 5.1 - A rule so simple a robot with three sensors can follow it

Behind the Scenes

The left-hand rule has an honest limitation, and stating it is part of good engineering. If the maze contains a loop that is not connected to the outer wall, and the exit sits inside that loop, a wall-follower will circle forever. Mazes with this property are called **non-simply-connected**. Competition mazes are usually designed to be solvable by wall-following - but you should say so in your report rather than claim your robot solves any maze.

Priority	If this is possible, do it	Why this order
1	Turn left	The rule: always prefer the left-hand wall
2	Go straight	Only when left is blocked
3	Turn right	Only when both left and straight are blocked
4	Turn around	A dead end - the last resort

Write your specification before Session 13. Four testable requirements is enough, and here is the shape they should take.

#	Requirement	How it will be tested
R1	The robot shall reach the exit of a 2 x 2 m maze without human intervention.	Three consecutive runs, no rescues
R2	The robot shall not touch a maze wall during a run.	Observation by a second group
R3	The second run shall be faster than the first.	Stopwatch, both runs recorded
R4	A run shall complete within _____ seconds.	Stopwatch

Role	Responsible for	Name
Systems lead	The specification, the test plan and the final report	_____
Mechanical	Chassis, sensor mounting height and spacing, wiring discipline	_____
Software	State machine, path storage, tuning	_____
Test engineer	Running trials, recording every result including failures	_____

SESSION 13

one class

Subsystem Build and Independent Testing**Today you will**

- Build the drive base and mount three IR sensors.
- Test each subsystem alone before integrating.
- Verify the direction table for both motors.

The single most valuable habit in this session is **independent testing**. Build the drive base and prove it drives. Then mount the sensors and prove they read correctly, with the motors switched off. Only then combine them. Integrate two untested subsystems and any fault has four possible homes instead of one.

How an IR sensor tells black from white

It shines invisible infrared light down and watches how much bounces back

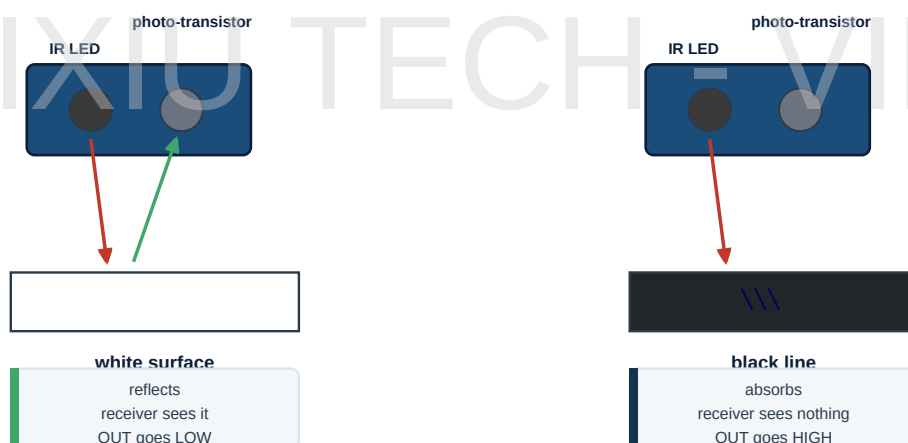


Figure 5.1 - Two tiny components, one very useful trick

Each IR module carries an infrared LED and a phototransistor. White scatters the light back and the output goes LOW; black absorbs it and the output goes HIGH. Because the sensor supplies its own illumination, it works identically in a dark room and a bright one - though direct sunlight can still saturate it.

13.1 The Power Stage

An Arduino output pin is a signalling device, not a power supply. It can deliver about 20 milliamps safely; a BO motor starting under load can demand several hundred. Connect one directly and the pin is destroyed permanently.

The L298N contains two **H-bridges**, each a set of four electronic switches arranged around a motor in the shape of the letter H. Close one diagonal pair and current flows one way through the motor; close the other pair and it flows the other way. The Arduino only ever sends tiny logic signals telling those switches what to do.

The L298N: two switches decide which way a motor turns

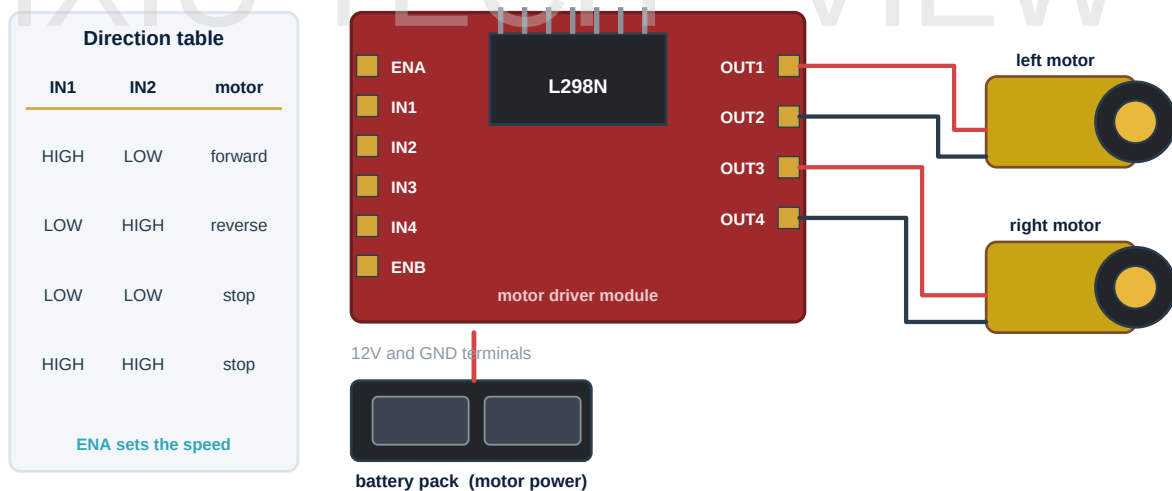


Figure 5.1 - IN pins choose the direction, EN pins choose the speed

Look at your BO motor too. The silver cylinder at the back is the motor itself, spinning several thousand times a minute with almost no strength. The plastic housing in front is a **gearbox** that trades that speed for torque - gear down by fifty and you turn fifty times slower with roughly fifty times the turning force. That is why the wheel runs at a usable 100 rpm and can actually push the chassis, battery and boards along the floor.

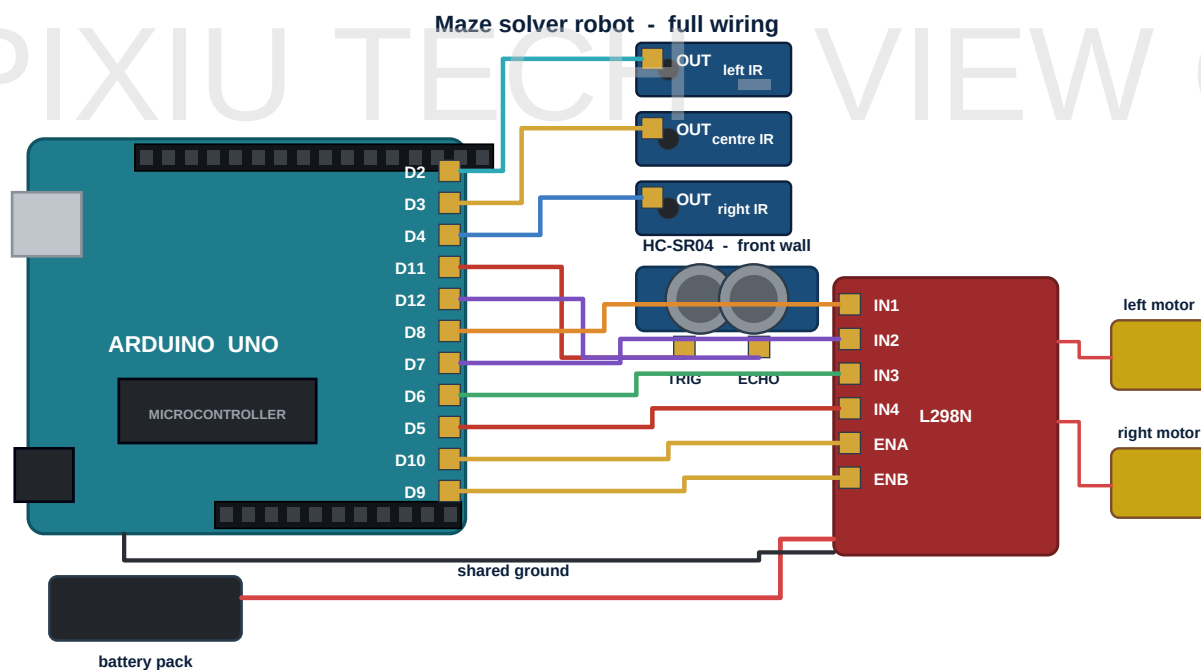


Figure 5.2 - Three line sensors, one wall sensor, two motors

Connection	Arduino pin	Note
Left IR OUT	D2	Mounted about 3 cm left of centre
Centre IR OUT	D3	Directly on the line
Right IR OUT	D4	About 3 cm right of centre
HC-SR04 TRIG / ECHO	D11 / D12	Front wall detection
L298N IN1 / IN2	D8 / D7	Left motor direction
L298N IN3 / IN4	D6 / D5	Right motor direction

Connection	Arduino pin	Note
L298N ENA / ENB	D10 / D9	PWM speed
L298N GND	Arduino GND	Shared ground - mandatory
L298N 12V / GND	Battery + / -	Motor supply, never from the Arduino

Watch Out

Remove the ENA and ENB jumper caps before wiring them, or the motors will run at full speed and ignore every speed value you write. And check the sensor mounting height: 5 to 10 mm above the floor. Too high and the reflection is too weak; too low and the sensor sees only its own glow.

SESSION 14

one class

State Machines and Path Memory**Today you will**

- Design the robot's behaviour as a state machine.
- Store the route taken as a sequence of turns.
- Run the maze for the first time.

A long chain of if statements becomes unmanageable once a machine has to remember what it was doing. The professional structure is a **state machine**: the robot is always in exactly one named state, each state has clear behaviour, and specific events move it to another state.

State	What the robot does	What moves it on
FOLLOWING	Steers to keep the centre sensor on the line	A junction is detected
AT_JUNCTION	Stops, reads all three sensors and the front wall	A decision is made
TURNING	Executes the chosen turn for a tuned time	The turn completes
DEAD_END	Turns around 180 degrees	The turn completes
FINISHED	Stops and reports the stored path	Nothing - the run is over

The memory is simpler than students expect. Every turn is recorded as a single character: L for left, R for right, S for straight, B for back. A route becomes a short string such as LBLIRS. Then comes the elegant part - that string can be **simplified**. Any sequence containing a B represents a dead end that was entered and left, and can be replaced by the single turn that would have avoided it.

What was walked	Becomes	In plain English
LBR	B	Went left, dead end, came back right - it was a dead end straight ahead
LBS	R	Left, dead end, straight on - should have turned right
RBL	B	Right, dead end, left - dead end ahead
SBS	B	Straight, dead end, straight - turn around
LBL	S	Left, dead end, left - should have gone straight

MazeSolver.ino - the state machine and the path memory

```
// ----- states -----
#define FOLLOWING 0
#define AT_JUNCTION 1
#define TURNING 2
#define FINISHED 3
```

```

int state = FOLLOWING;

char path[40];           // the route, one character per turn
int pathLength = 0;      // watch your SRAM - 40 is plenty

int leftIR = 2, centreIR = 3, rightIR = 4;
int in1 = 8, in2 = 7, in3 = 6, in4 = 5, ena = 10, enb = 9;
int speed = 120;

void recordTurn(char c) {
    if (pathLength < 39) {
        path[pathLength] = c;
        pathLength = pathLength + 1;
    }
}

void simplify() {        // collapse a dead end into one turn
    if (pathLength < 3) { return; }
    if (path[pathLength - 2] != 'B') { return; }

    char a = path[pathLength - 3];
    char c = path[pathLength - 1];
    char result = 'S';

    if (a == 'L' && c == 'R') { result = 'B'; }
    if (a == 'L' && c == 'S') { result = 'R'; }
    if (a == 'R' && c == 'L') { result = 'B'; }
    if (a == 'S' && c == 'S') { result = 'B'; }
    if (a == 'L' && c == 'L') { result = 'S'; }

    pathLength = pathLength - 3;
    recordTurn(result);
}

void loop() {
    int L = digitalRead(leftIR);
    int C = digitalRead(centreIR);
    int R = digitalRead(rightIR);

    switch (state) {

        case FOLLOWING:
            followLine(L, C, R);
            if (L == HIGH || R == HIGH) { state = AT_JUNCTION; }
            break;

        case AT_JUNCTION:
            stopMotors();
            delay(120);           // let the chassis settle
            if (L == HIGH)        { recordTurn('L'); turnLeft(); }
            else if (C == HIGH)   { recordTurn('S'); goStraight(); }
            else if (R == HIGH)   { recordTurn('R'); turnRight(); }
            else                   { recordTurn('B'); turnAround(); }
            simplify();
            state = TURNING;
            break;

        case TURNING:
            if (C == HIGH) { state = FOLLOWING; } // back on the line
            break;

        case FINISHED:
            stopMotors();
            break;
    }
}

```

Behind the Scenes

Notice that `simplify()` runs after every turn, not once at the end. That keeps the stored path short as you go, which matters when you only have two kilobytes of SRAM. It is a small design decision with a real engineering reason behind it - exactly the kind of thing a viva examiner will ask you to justify.

SESSION 15

one class

Optimisation, Final Runs and Technical Viva**Today you will**

- Use the stored path to run the maze a second time, faster.
- Complete the acceptance tests and record the data.
- Defend your design in a technical viva.

Once the exit is reached, the simplified path holds the direct route with every dead end removed. On the second run the robot no longer needs to think at each junction - it simply reads the next character and executes it. That is why the second run is faster, and it is the requirement R3 you wrote in Session 12.

The second run: no decisions, just recall

```
int step = 0;

void fastRun() {
  if (atJunction()) {
    char next = path[step];
    step = step + 1;

    if (next == 'L') { turnLeft(); }
    else if (next == 'R') { turnRight(); }
    else if (next == 'S') { goStraight(); }

    if (step >= pathLength) { state = FINISHED; }
  } else {
    followLine(digitalRead(leftIR), digitalRead(centreIR), digitalRead(rightIR));
  }
}
```

What you see	Most likely cause	What to do
Nothing happens	Missing Arduino-to-L298N ground	Add it. The commonest fault.
Speed value ignored	ENA and ENB caps still fitted	Remove both
Turns overshoot or undershoot	Turn duration not tuned for this floor	Adjust in 50 ms steps, retest three times each
Junction detected twice	Sensor still over the marking after the turn	Add a short delay after a turn completes
Path grows and never simplifies	The B character is not being recorded	Print the path after every turn and watch it
Robot drifts on straights	Motors mismatched	Trim one motor's PWM by 5 to 10
Acceptance test	Result	Tick
R1 Reaches the exit unaided, three consecutive runs	_____	■
R2 No wall contact during a run	_____	■

Acceptance test	Result	Tick
R3 Second run measurably faster than the first	_____	■
R4 Run completes within the stated time	_____	■
Stored path simplifies correctly, printed and checked	_____	■

15.1 The Technical Viva

You will be asked to defend your design, not merely demonstrate it. Prepare honest answers to these, with data where data exists.

- 1 Why did you choose the left-hand rule, and what kind of maze would defeat it?
- 2 Show the state machine and explain why you did not use a chain of if statements.
- 3 Explain one path simplification rule using a route your robot actually walked.
- 4 What is your robot's slowest and fastest recorded run, and what accounts for the difference?
- 5 Which subsystem failed most often during testing, and how did you diagnose it?
- 6 If you had one more session, what would you change, and what evidence supports that choice?

Challenge

Add the HC-SR04 as a genuine safety layer. Before executing any forward movement, check the distance ahead and refuse to move if a wall is closer than 8 cm. Log every refusal to the serial line. Real autonomous systems all have a layer like this, and it is called a safety interlock.

New word	What it means
State machine	A design where the system is always in one named state.
Left-hand rule	Always prefer the leftmost available turn.
Path simplification	Collapsing a dead-end detour into the single turn that avoids it.
Safety interlock	A layer that refuses an unsafe action regardless of the main logic.
Independent testing	Proving each subsystem alone before integrating.